

QoS Policies – Purple Team

 ipurple.team/2026/06/17/qos-policies

June 17, 2026

In Windows, a Quality of Service (QoS) policy is a rule that handles outbound network traffic. Specifically, it is used to cap the outbound bandwidth of a process, port, or protocol. Organizations can configure QoS policies through Group Policies, MDM, or PowerShell. Threat actors with elevated privileges on the asset can point a QoS policy at an EDR agent and set the rate to effectively zero, limiting the capability of the agent to generate telemetry in the cloud console.

Playbook

Execution of the following command will create a new QoS policy to limit the bandwidth of the FTP application via PowerShell:

```
1 New-NetQosPolicy -Name "FTP" -AppPathNameMatchCondition "ftp.exe" -  
ThrottleRateActionBitsPerSecond 1MB -PolicyStore ActiveStore
```

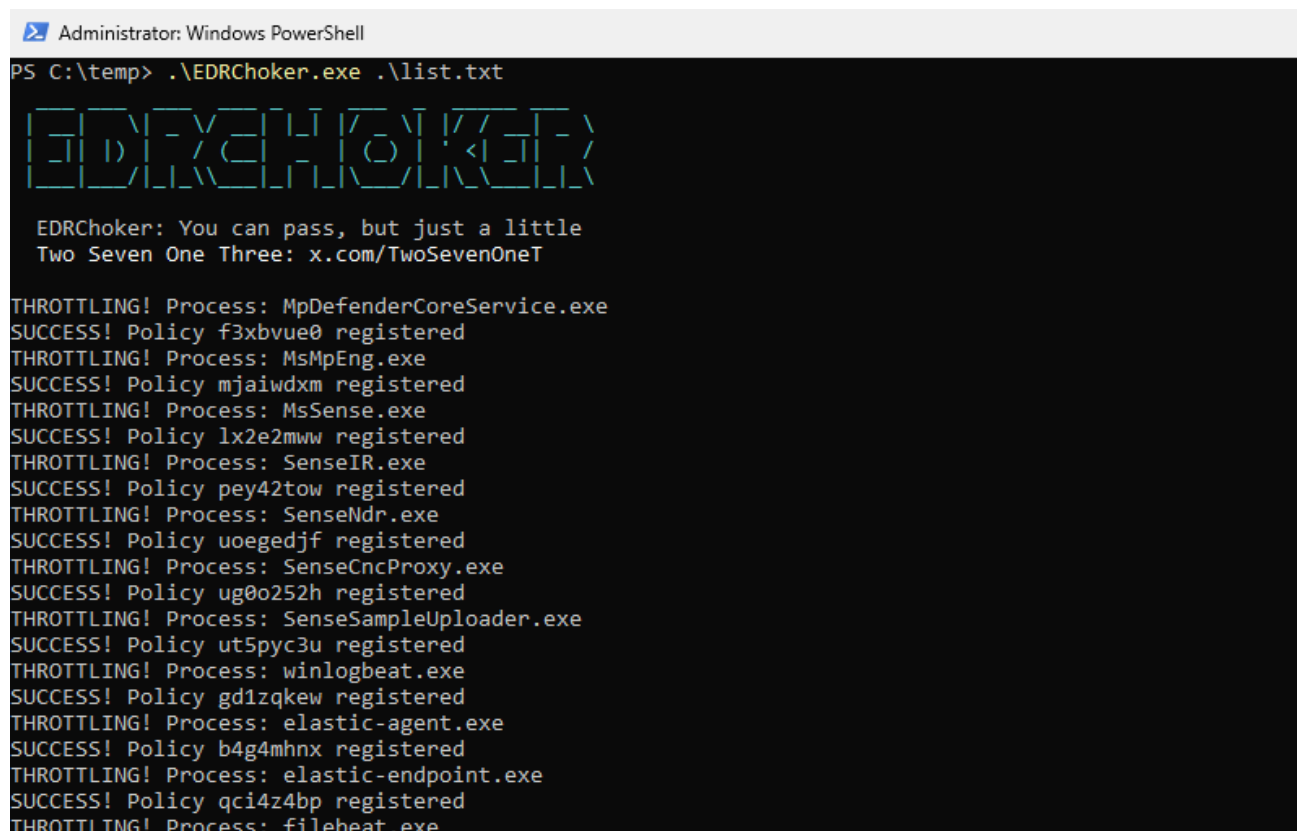


Add New QoS Policy

The `ThrottleRateActionBitsPerSecond` parameter defines the maximum outbound bandwidth limit for traffic that matches the application or the protocol. [Grimur Grimursson](#)

demonstrated in an article that it is possible to abuse the *ThrottleRateActionBitsPerSecond* to prevent telemetry generated in the EDR cloud console. It should be noted that the technique doesn't impair EDR rules, since some EDRs like CrowdStrike have their rules, stack deployed locally in the endpoints. Setting the *ThrottleRateActionBitsPerSecond* to 1KBps limits bandwidth to 8,000 bits per second disrupting Internet connectivity for many modern applications like EDR. A normal TLS 1.3 HTTPS connection with server authentication only, requires 3-6 KB for a small certificate chain and 6-15 KB for a larger certificate chain. This will cause the EDR agent to timeout since it will not be possible to complete the full TLS handshake due to the bandwidth limitation. The technique is different compared to the EDR Silencing because packets are not dropped but limited due to the very low bandwidth.

A proof of concept was released called [EDRChoker](#) that takes a list of common EDR process names and creates QoS policies that restrict the bandwidth of these processes to 8 bits per second. It should be noted that Administrator-level privileges are required to create Quality of Service policies.



```
Administrator: Windows PowerShell
PS C:\temp> .\EDRChoker.exe .\list.txt

EDRCHOKER

EDRChoker: You can pass, but just a little
Two Seven One Three: x.com/TwoSevenOneT

THROTTLING! Process: MpDefenderCoreService.exe
SUCCESS! Policy f3xbvue0 registered
THROTTLING! Process: MsMpEng.exe
SUCCESS! Policy mjaiwdxm registered
THROTTLING! Process: MsSense.exe
SUCCESS! Policy lx2e2mww registered
THROTTLING! Process: SenseIR.exe
SUCCESS! Policy pey42tow registered
THROTTLING! Process: SenseNdr.exe
SUCCESS! Policy uoegedjf registered
THROTTLING! Process: SenseCncProxy.exe
SUCCESS! Policy ug0o252h registered
THROTTLING! Process: SenseSampleUploader.exe
SUCCESS! Policy ut5pyc3u registered
THROTTLING! Process: winlogbeat.exe
SUCCESS! Policy gd1zqkew registered
THROTTLING! Process: elastic-agent.exe
SUCCESS! Policy b4g4mhnx registered
THROTTLING! Process: elastic-endpoint.exe
SUCCESS! Policy qci4z4bp registered
THROTTLING! Process: filebeat.exe
```

EDRChoker – Proof of Concept

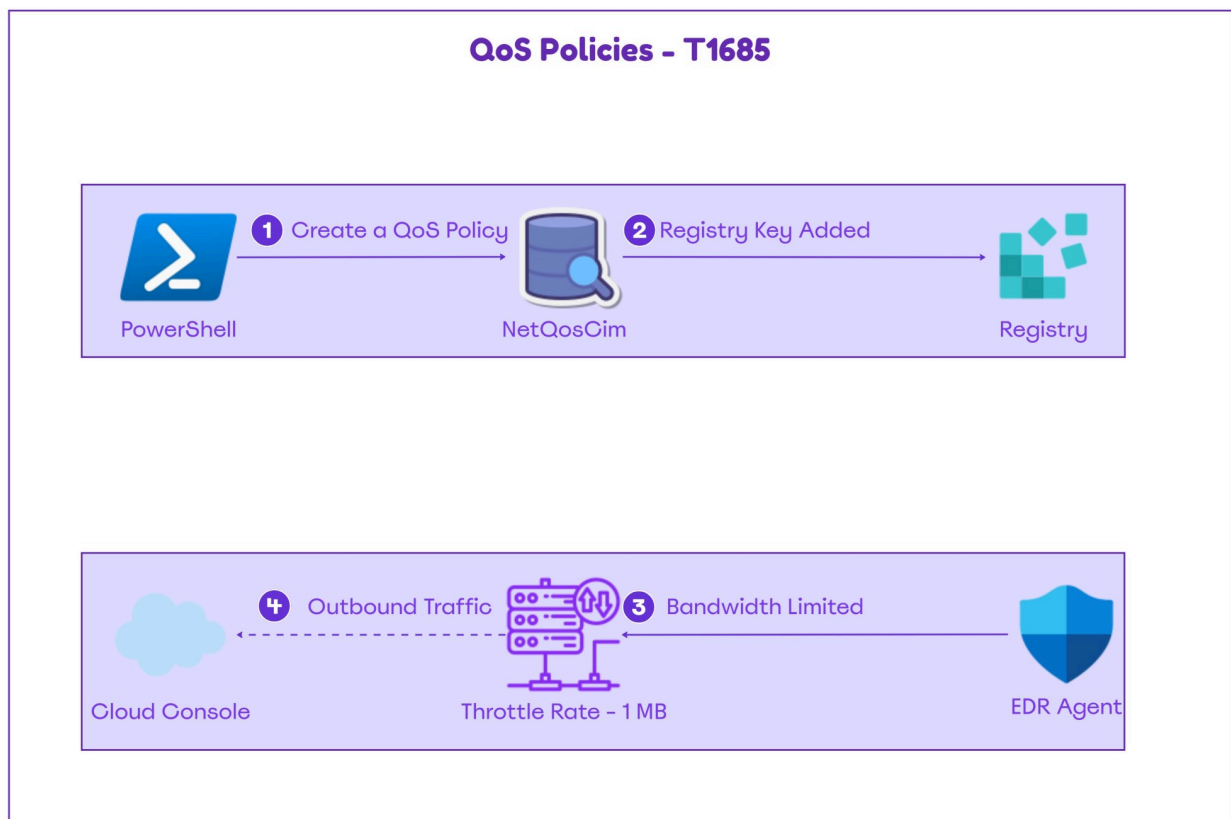
According to the code, the proof-of-concept creates the Quality-of-Service policy using WMI. Initially, a connection is established to the CIM namespace where the QoS provider is stored and binds to the *NetQoSPolicySettingData* class. A new policy object is created, and the owner is set to 1. This is equivalent to the `New-NetQoSPolicy -PolicyStore ActiveStore` command. The policy name is also randomized using 8 characters length.

```

1  static void CreateThrottleCurlPolicyPureWmi(string procName)
2  {
3  try
4  {
5  var scope = new ManagementScope(@"\\.\R00T\StandardCimv2");
6  scope.Connect();
7  var managementPath = new
8  ManagementPath("MSFT_NetQosPolicySettingData");
9  var policyClass = new ManagementClass(scope, managementPath, null);
10 ManagementObject newPolicy = policyClass.CreateInstance();
11 newPolicy["Owner"] = 1;
12 string guid = Guid.NewGuid().ToString();
13 string policyName = Path.GetRandomFileName().Replace(".",
14 "").Substring(0, 8);
15 newPolicy["Name"] = policyName;
16
17
18

```

The following diagram illustrates how QoS policies can be used to restrict EDR agents reporting back to the cloud console:



QoS Policies – Diagram

The technique abstract is displayed below:

MITRE ID - T1685	
Data Sources	Data Components
API	
Process	
Windows Registry	HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\QoS
Log Sources	4688 4663 5857 4104
DLL	qoswmi.dll
Network Protocol	

Technique Abstract

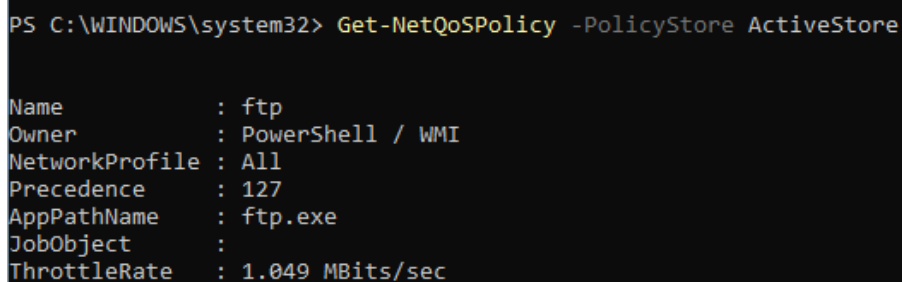
The playbook of the technique can be found below:

```
1  [[Playbook.QoS Policies]]
2  id = "1.0.0"
3  name = "1.0.0 - QoS Policies"
4  description = "EDR agent communication disruption via QoS Policies"
5  tooling.name = "EDRChoker"
6  tooling.references = [
7    "https://github.com/TwoSevenOneT/EDRChoker"
8  ]
9  executionSteps = [
10 "New-NetQosPolicy -Name "ipurple" -AppPathNameMatchCondition
11 "ipurple.exe" -ThrottleRateActionBitsPerSecond 1MB -PolicyStore
12 ActiveStore"
13 ]
14 executionRequirements = [
    "Local Administrator"
  ]
```

Detection

The technique of abusing QoS policies to restrict EDR agents from communicating back to their cloud server introduces multiple challenges for cyber defense teams. Threat actors could directly abuse the `New-NetQoSPolicy` cmdlet or utilize WMI to create new policies on the asset. Therefore, there is no need to introduce new binaries on the host. The other challenge is that EDR agents might continue to appear as healthy. It is recommended that SOC teams should not rely on EDR queries to engineer detections, but focus on other suspicious patterns, especially if QoS policies are not used in their environments. Non-persistent Quality of Service policies are stored in *ActiveStore*, and SOC teams could use the `Get-NetQoSPolicy` cmdlet to review policies either proactively or reactively during an incident.

```
1 Get-NetQoSPolicy -PolicyStore ActiveStore
```



```
PS C:\WINDOWS\system32> Get-NetQoSPolicy -PolicyStore ActiveStore

Name           : ftp
Owner          : PowerShell / WMI
NetworkProfile : All
Precedence     : 127
AppPathName    : ftp.exe
JobObject      :
ThrottleRate   : 1.049 MBits/sec
```

Retrieve QoS Policies – ActiveStore

For hosts that might contain multiple policies, the following PowerShell code could be used to format the output in a table using only the *Name*, *PolicyStore*, *AppPathName*, and *ThrottleRate*.

```
1 $stores = @(
2   "ActiveStore"
3 )
4 $results = foreach ($store in $stores) {
5   try {
6     Get-NetQoSPolicy -PolicyStore $store | Select-Object *,
7     @{Name="PolicyStore";Expression={$store}}
8   } catch {}
9 }
10 $results | Sort-Object PolicyStore, Name | Format-Table Name,
11 PolicyStore, AppPathNameMatchCondition,
12 ThrottleRateActionBitsPerSecond
```

```

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> $stores = @(
>> "ActiveStore"
>> )
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $results = foreach ($store in $stores) {
>>     try {
>>         Get-NetQoSPolicy -PolicyStore $store | Select-Object *,
>>         @{Name="PolicyStore";Expression={$store}}
>>     } catch {}
>> }
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $results | Sort-Object PolicyStore, Name | Format-Table Name, PolicyStore, AppPathNameMatchCondi
tion, ThrottleRateActionBitsPerSecond

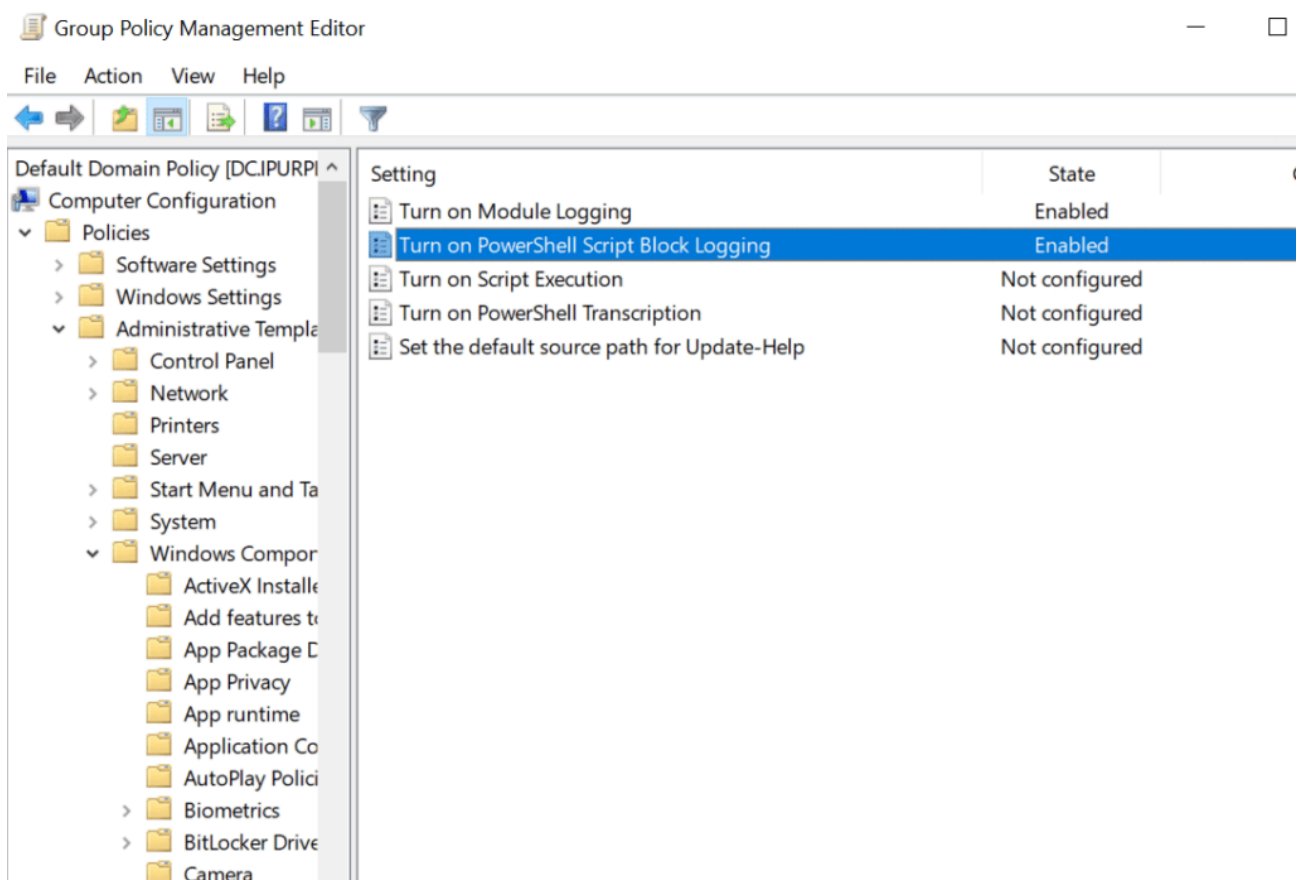
Name PolicyStore AppPathNameMatchCondition ThrottleRateActionBitsPerSecond
-----
ftp ActiveStore ftp.exe

PS C:\WINDOWS\system32> _

```

Format QoS Policies – ActiveStore

The technique has two methods of execution: PowerShell and WMI. However, the PowerShell cmdlet is just a wrapper for the WMI execution. Organizations should ensure that the required visibility is enabled to ensure PowerShell activities are captured in the logs. From the Group Policy, the settings *Turn on Module Logging* and *Turn on PowerShell Script Block Logging* should be enabled.



PowerShell Logging

The following command will create a new QoS policy under the name ipurple for testing purposes:

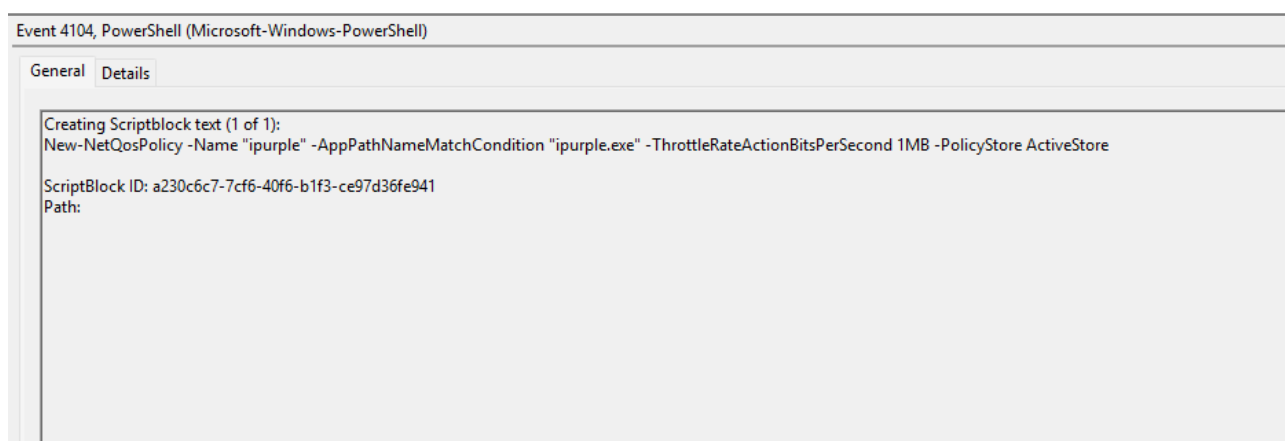
```
1 New-NetQosPolicy -Name "ipurple" -AppPathNameMatchCondition "ipurple.exe"
  -ThrottleRateActionBitsPerSecond 1MB -PolicyStore ActiveStore
```

```
PS C:\WINDOWS\system32> New-NetQosPolicy -Name "ipurple" -AppPathNameMatchCondition "ipurple.exe" -ThrottleRateActionBitsPerSecond 1MB -PolicyStore ActiveStore

Name           : ipurple
Owner          : PowerShell / WMI
NetworkProfile : All
Precedence     : 127
AppPathName    : ipurple.exe
JobObject      :
ThrottleRate   : 1.049 MBits/sec
```

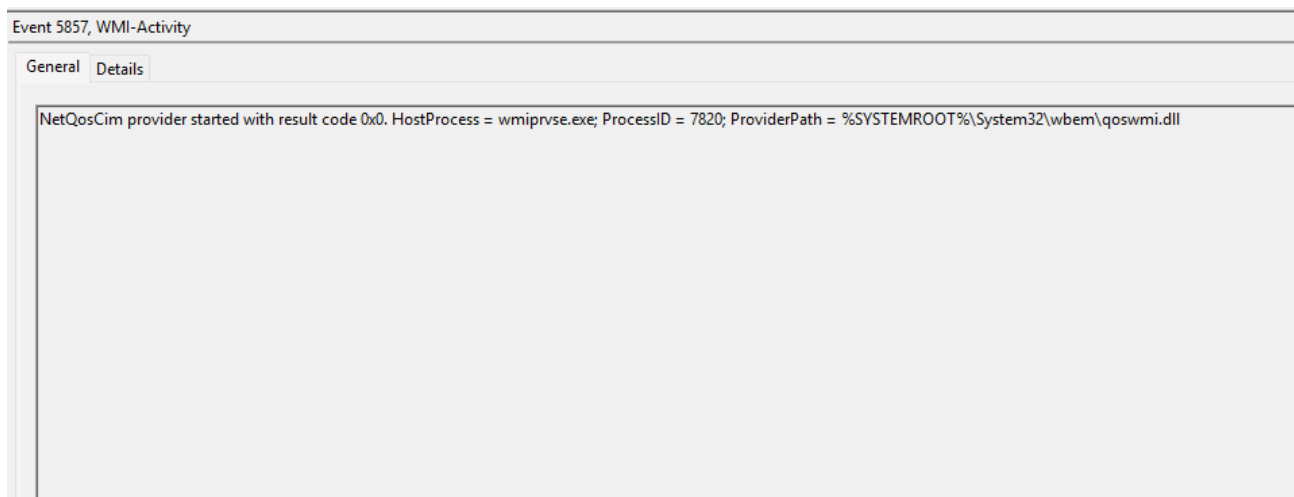
Add New QoS Policy – iPurple

The execution will be captured under the PowerShell event ID 4104. In environments that don't utilize QoS policies, SOC teams should treat execution of this command as malicious. However, if QoS policies are used, these should be officially recorded and approved to assist with correlation during threat hunting activities. The field *ThrottleRateActionBitsPerSecond*, which defines the outbound traffic bandwidth, is critical for distinguishing arbitrary QoS policies from legitimate ones.



PowerShell ScriptBlock – Event ID 4104

It should be noted that there is no generation of event ID 4103 because the *New-NetQoSPolicy* is a binary CIM cmdlet that performs no PowerShell pipeline script execution. Another observed behaviour is the loading of the *qoswmi.dll* into the *WmiPrvSE.exe* process. This activity is generated when the *NetQoSPolicy* cmdlet touches the CIM layer to initiate the *NetQoS Cim* provider. The event ID 5857 is common between both PowerShell and WMI executions. SOC teams should use this event ID as an indicator of QoS policies abuse.



NetQosCim Provider

The following PowerShell snippet can be used to query WMI logs that match the *NetQosCim* provider and *qoswmi.dll*.

```
1  Get-WinEvent -LogName 'Microsoft-Windows-WMI-Activity/Operational' -
2  MaxEvents 5000 |
3  Where-Object { $_.Id -eq 5857 -and $_.Message -match
4  'NetQosCim|qoswmi\.dll' } |
5  ForEach-Object {
6  $x = [xml]$_ToXml()
7  [pscustomobject]@{
8  Time          = $_.TimeCreated
9  Provider      =
10 $x.Event.UserData.Operation_StartedOperational.ProviderName
11 HostProcess   =
12 $x.Event.UserData.Operation_StartedOperational.HostProcess
   WmiPrvSE_PID =
   $x.Event.UserData.Operation_StartedOperational.ProcessID
   ProviderPath =
   $x.Event.UserData.Operation_StartedOperational.ProviderPath
   }
   } | Sort-Object Time -Descending | Format-Table -Auto
```

```

PS C:\Windows\System32> Get-WinEvent -LogName 'Microsoft-Windows-WMI-Activity/Operational' -MaxEvents 5000 |
>> Where-Object { $_.Id -eq 5857 -and $_.Message -match 'NetQosCim\qoswmi\.dll' } |
>> ForEach-Object {
>>     $x = [xml]$_ .ToXml()
>>     [pscustomobject]@{
>>         Time           = $_.TimeCreated
>>         Provider       = $x.Event.UserData.Operation_StartedOperational.ProviderName
>>         HostProcess    = $x.Event.UserData.Operation_StartedOperational.HostProcess
>>         WmiPrvSE_PID   = $x.Event.UserData.Operation_StartedOperational.ProcessID
>>         ProviderPath   = $x.Event.UserData.Operation_StartedOperational.ProviderPath
>>     }
>> } | Sort-Object Time -Descending | Format-Table -Auto

```

Time	Provider	HostProcess	WmiPrvSE_PID	ProviderPath
6/15/2026 8:23:43 PM	NetQosCim	wmiprvse.exe	4244	%SYSTEMROOT%\System32\wbem\qoswmi.dll
6/15/2026 8:20:17 PM	NetQosCim	wmiprvse.exe	7568	%SYSTEMROOT%\System32\wbem\qoswmi.dll
6/15/2026 8:18:32 PM	NetQosCim	wmiprvse.exe	7612	%SYSTEMROOT%\System32\wbem\qoswmi.dll

PowerShell Query – ETW WMI

An alternative method is to enumerate all active QoS policies is to query the QoS policy settings using a WMI query.

- 1 `Get-CimInstance -Namespace root/StandardCimv2 -ClassName MSFT_NetQosPolicySettingData`

```

Administrator: Windows PowerShell
PS C:\temp> Get-CimInstance -Namespace root/StandardCimv2 -ClassName MSFT_NetQosPolicySettingData

```

Name	: 0begslgv
Owner	: Group Policy (Machine)
NetworkProfile	: All
Precedence	: 127
AppPathName	: sfc.exe
JobObject	:
IPProtocol	: Both
ThrottleRate	: 8 Bits/sec
Name	: 0exxgmr
Owner	: Group Policy (Machine)
NetworkProfile	: All
Precedence	: 127
AppPathName	: TMBMSRV.exe
JobObject	:
IPProtocol	: Both
ThrottleRate	: 8 Bits/sec
Name	: 0ms0lw4z
Owner	: Group Policy (Machine)
NetworkProfile	: All
Precedence	: 127
AppPathName	: SentinelBrowserNativeHost.exe
JobObject	:
IPProtocol	: Both
ThrottleRate	: 8 Bits/sec

Query WMI NetQosPolicySettingData

The output could be formatted to target only specific fields:

- 1 `Get-CimInstance -Namespace root/StandardCimv2 -ClassName MSFT_NetQosPolicySettingData | Select-Object Name, Owner, AppPathNameMatchCondition, DSCPAction, ThrottleRateAction`

```
Administrator: Windows PowerShell
PS C:\temp> Get-CimInstance -Namespace root/StandardCimv2 -ClassName MSFT_NetQosPolicySettingData |
>> Select-Object Name, Owner, AppPathNameMatchCondition, DSCPAction, ThrottleRateAction

Name           : 0begslgv
Owner          : Group Policy (Machine)
AppPathNameMatchCondition : sfc.exe
DSCPAction     : -1
ThrottleRateAction : 8

Name           : 0exxgmr
Owner          : Group Policy (Machine)
AppPathNameMatchCondition : TMBMSRV.exe
DSCPAction     : -1
ThrottleRateAction : 8

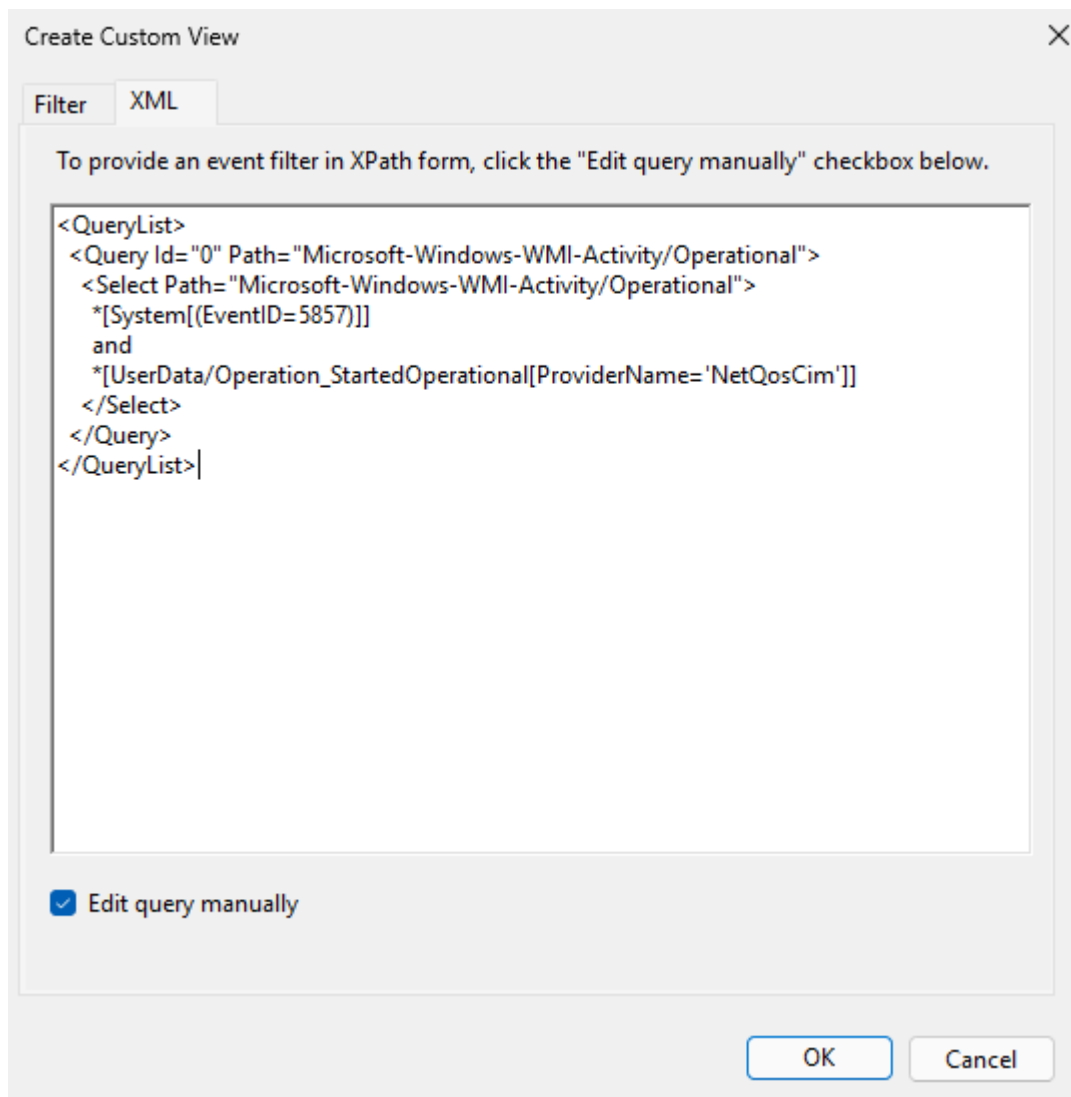
Name           : 0ms0lw4z
Owner          : Group Policy (Machine)
AppPathNameMatchCondition : SentinelBrowserNativeHost.exe
DSCPAction     : -1
ThrottleRateAction : 8

Name           : 0pvp4c4j
Owner          : Group Policy (Machine)
AppPathNameMatchCondition : QualysAgent.exe
DSCPAction     : -1
ThrottleRateAction : 8
```

Format NetQosPolicySettingData

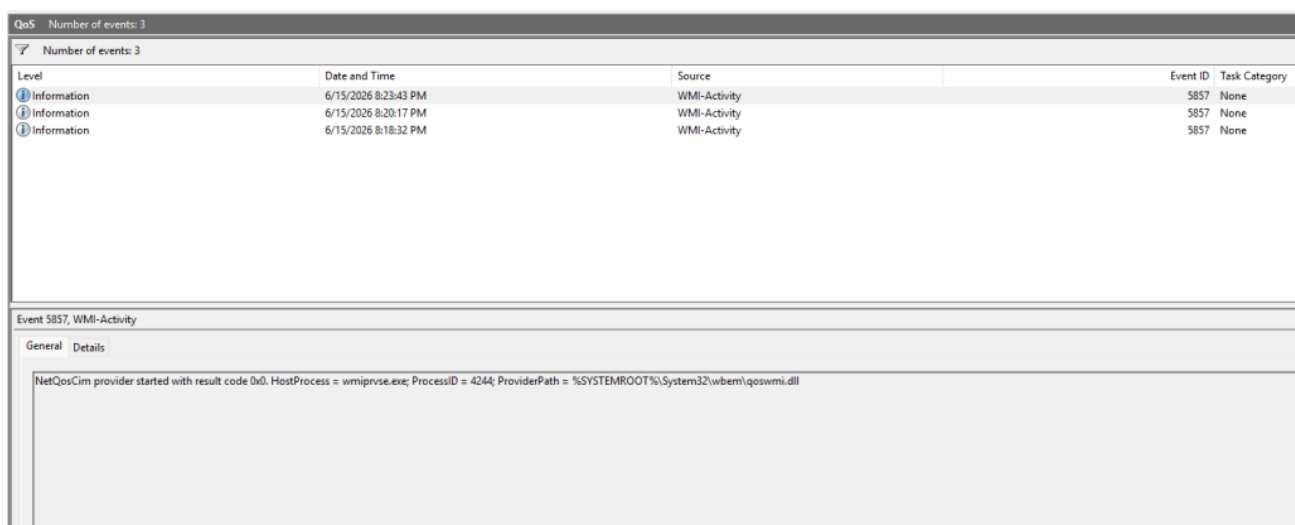
SOC teams could use the following Windows Event Filter (WEF) to collect only the relevant 5857 event IDs associated with *NetQosCim* provider to reduce the volume size of WMI events.

```
1 <QueryList>
2 <Query Id="0" Path="Microsoft-Windows-WMI-Activity/Operational">
3 <Select Path="Microsoft-Windows-WMI-Activity/Operational">
4 *[System[(EventID=5857)]]
5 and
6 *[UserData/Operation_StartedOperational[ProviderName='NetQosCim']]
7 </Select>
8 </Query>
9 </QueryList>
```



Event Filter

When the filter is loaded in the Windows Events, only the relevant logs will be displayed.



Event Filter – Output

[Samir Bousseaden](#) released a fileless WMI remediation called [EDRUnChoker](#) that registers a permanent subscription in root\subscription. The tool enumerates every 5 seconds QoS policies on *ActiveStore* and GPO, and attempts to remove arbitrary policies.

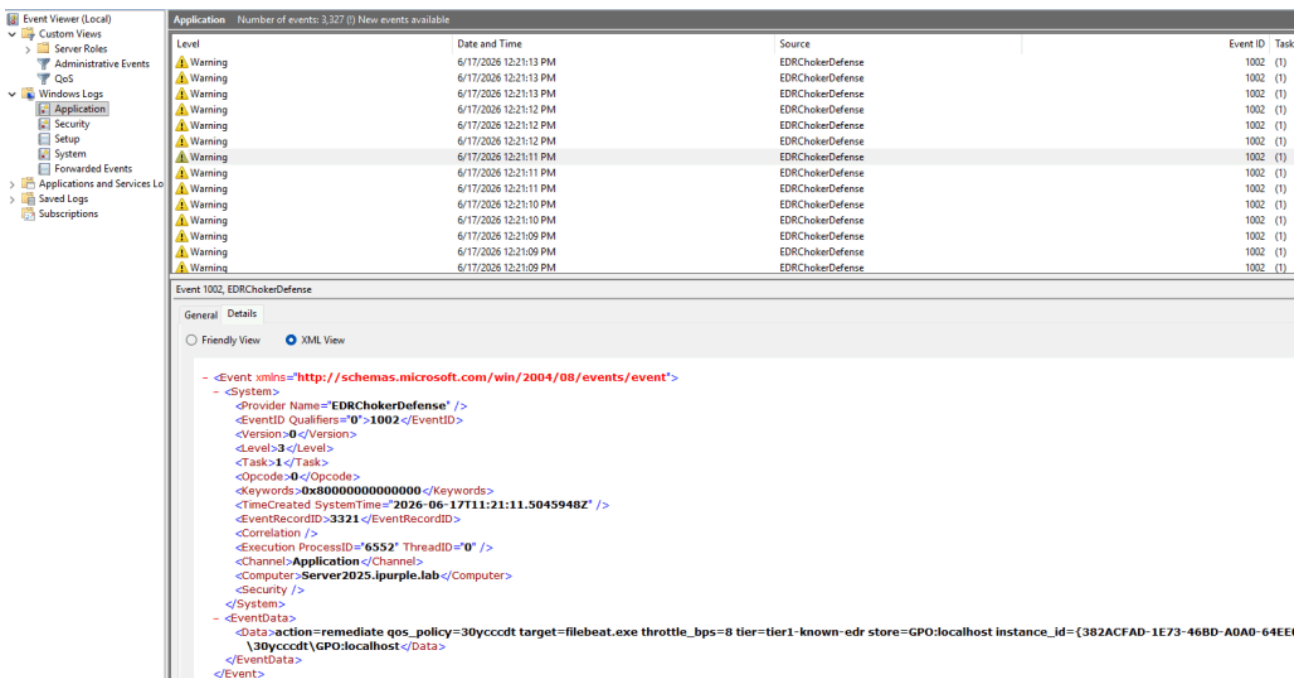
```
1 .\Install-EdrChokerWmiDefense.ps1
```

```
Administrator: Command Prompt - powershell
PS C:\temp> .\Install-EdrChokerWmiDefense.ps1
Fileless WMI subscription installed (hardened detection, polls every 5 s).
Timer      : EDRChokerDefense_Timer (5000ms)
Filter     : EDRChokerDefense_QoSFilter
Consumer   : EDRChokerDefense_QoSConsumer (ActiveScriptEventConsumer / VBScript)
Detection  : WbemContext PolicyStore ActiveStore + GPO:localhost
Event log  : Application / Source EDRChokerDefense (ID 1002 = remediate, 1003 = failed)

Reinstall required when upgrading embedded remediation logic.
Verify: .\Get-EdrChokerDefenseStatus.ps1
SOC query : Get-WinEvent -FilterHashtable @{ LogName='Application'; ProviderName='EDRChokerDefense' } -MaxEvents 20
PS C:\temp>
```

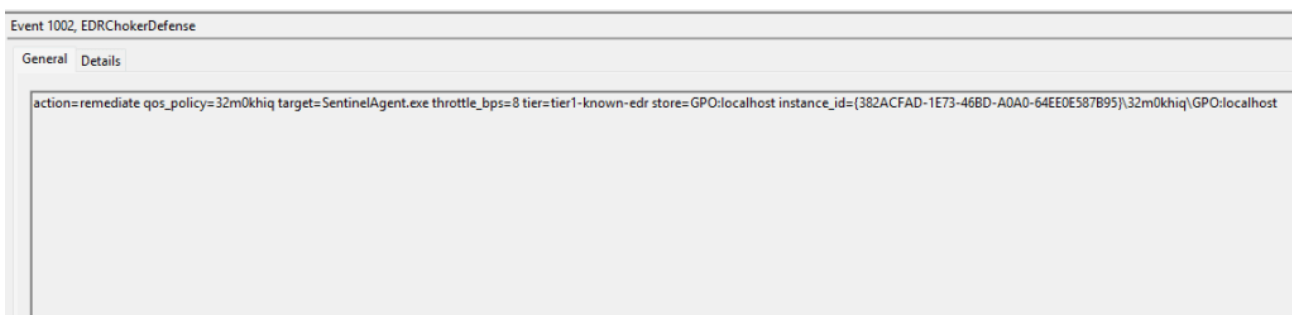
EDRUnChoker – Installation

The subscription is visible in the Application logs using the data source EDRChokerDefense.



EDRUnChoker – Subscription Event IDs

The event ID 1002 captures the malicious QoS policies to aid investigations. The tool contains a known list of EDR processes hardcoded to match the policy with the application name and identify malicious entries.



EDRUnChoker – Event ID

The malicious policies are also displayed in the console. The tool uses two conditions to flag a policy as malicious: Known process names of EDR agents and aggressive throttle rates of ≤ 1 Mbps.

```

=== Policies matching hardened detection rules ===
Count: 153 / 153 total QoS policies

Name      AppPath      Throttle Owner
-----
0begslgv  sfc.exe      8 Group Policy (Machine)
0exxgmo  tmbmsrv.exe  8 Group Policy (Machine)
0ms0lw4z  sentinelbrowsernativehost.exe  8 Group Policy (Machine)
0pvp4c4j  qualysagent.exe  8 Group Policy (Machine)
0shh1w5h  hurukai.exe  8 Group Policy (Machine)
1jqcwt1m  endpointbasecamp.exe  8 Group Policy (Machine)
1oevbl25  cybereasonav.exe  8 Group Policy (Machine)
1wy0crtj  tmlisten.exe  8 Group Policy (Machine)
1z1ojwfl  taniumclient.exe  8 Group Policy (Machine)
211sjaiz  qualysagent.exe  8 Group Policy (Machine)
23rzo2pv  cntaasmgr.exe  8 Group Policy (Machine)
2as1eerk  traps.exe  8 Group Policy (Machine)
2diqogs3  ntrtscan.exe  8 Group Policy (Machine)
2gfqq1  cbdefense.exe  8 Group Policy (Machine)
2ne133cv  sentinelstaticengine.exe  8 Group Policy (Machine)
2ph35nns  crssvc.exe  8 Group Policy (Machine)
2qbxmxgh  executionpreventionsvc.exe  8 Group Policy (Machine)
2rsykkah  taniumclient.exe  8 Group Policy (Machine)
2snec5ud  taniumdetectengine.exe  8 Group Policy (Machine)
2uafwyaq  sentinelagentworker.exe  8 Group Policy (Machine)
30ycccdt  filebeat.exe  8 Group Policy (Machine)
32m0khiq  sentinelagent.exe  8 Group Policy (Machine)
3o2qqj00  cyvrfsflt.exe  8 Group Policy (Machine)
3xodugit  fortiedr.exe  8 Group Policy (Machine)

```

Policies Matching Detection Rules

SOC teams can use the following PowerShell command to query events from the provider registered by the proof of concept on the asset.

```

1 Get-WinEvent -FilterHashtable @{ LogName='Application';
  ProviderName='EDRChokerDefense' } -MaxEvents 50

```

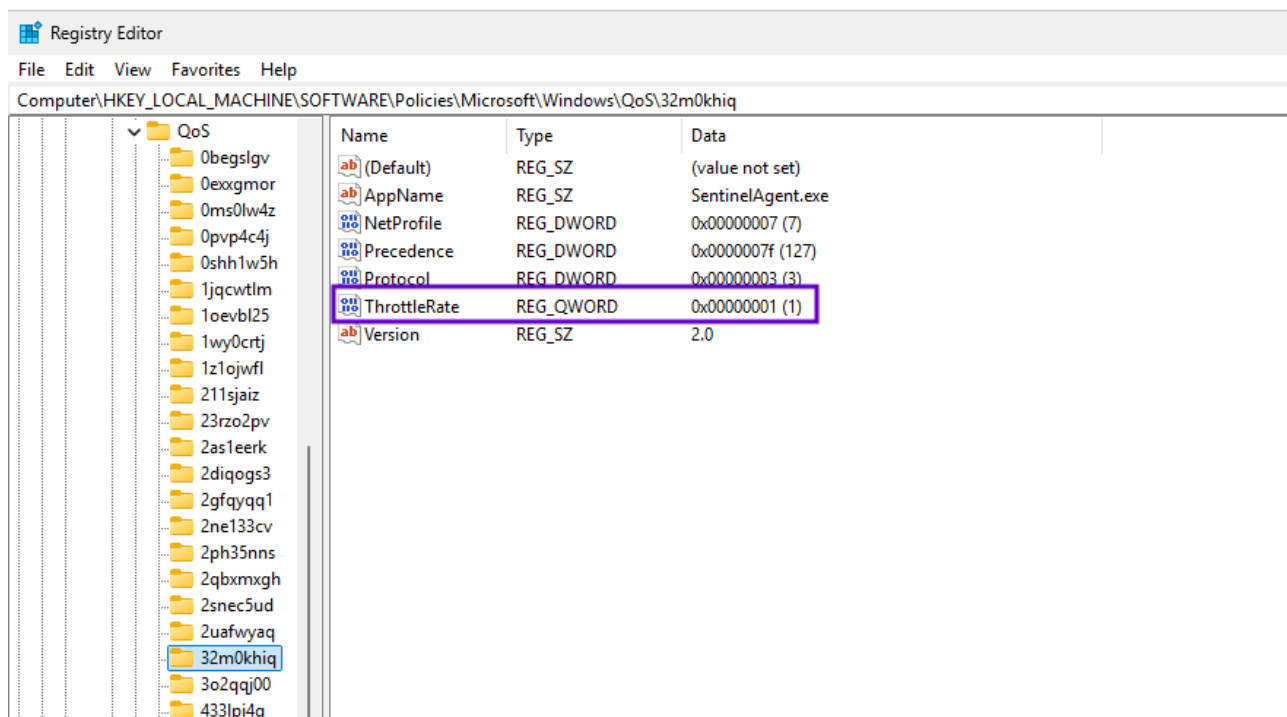
```
PS C:\Windows\System32> Get-WinEvent -FilterHashtable @{ LogName='Application'; ProviderName='EDRChokerDefense' } -MaxEvents 50

ProviderName: EDRChokerDefense

TimeCreated      Id LevelDisplayName Message
-----
6/17/2026 12:32:04 PM 1002 Warning action=remediate qos_policy=nnar220o target=sentinelhelperservi...
6/17/2026 12:32:04 PM 1002 Warning action=remediate qos_policy=nf2prtw target=cramtray.exe thrott...
6/17/2026 12:32:03 PM 1002 Warning action=remediate qos_policy=n1du5fki target=tmwscsvc.exe thrott...
6/17/2026 12:32:03 PM 1002 Warning action=remediate qos_policy=n33we5cm target=sentinelstaticengin...
6/17/2026 12:32:03 PM 1002 Warning action=remediate qos_policy=nlbdlxaz target=amsvc.exe throttle...
6/17/2026 12:32:03 PM 1002 Warning action=remediate qos_policy=nktj1z1r target=tmccsf.exe throttle...
6/17/2026 12:32:03 PM 1002 Warning action=remediate qos_policy=mzc0t20f target=sentinelagent.exe t...
6/17/2026 12:32:03 PM 1002 Warning action=remediate qos_policy=mavf10gc target=cylancesvc.exe thro...
6/17/2026 12:32:02 PM 1002 Warning action=remediate qos_policy=lx2e2mww target=mssense.exe throttl...
6/17/2026 12:32:02 PM 1002 Warning action=remediate qos_policy=mjaiwdxm target=msmpeng.exe throttl...
6/17/2026 12:32:02 PM 1002 Warning action=remediate qos_policy=mmvhn4ri target=sentinelServicehost...
6/17/2026 12:32:02 PM 1002 Warning action=remediate qos_policy=lxqdtgfl target=cyvrfsflt.exe throt...
6/17/2026 12:32:02 PM 1002 Warning action=remediate qos_policy=l00n0t14 target=ntrtscan.exe throttl...
6/17/2026 12:32:02 PM 1002 Warning action=remediate qos_policy=m02tjzm4 target=fortiedr.exe thrott...
6/17/2026 12:32:01 PM 1002 Warning action=remediate qos_policy=kspu4awu target=ekrn.exe throttle_b...
6/17/2026 12:32:01 PM 1002 Warning action=remediate qos_policy=klkau30h target=cntaasmgr.exe throt...
6/17/2026 12:32:01 PM 1002 Warning action=remediate qos_policy=jz4x4lez target=logprocessorservice...
6/17/2026 12:32:00 PM 1002 Warning action=remediate qos_policy=jtlpnvws target=hurukai.exe throttl...
6/17/2026 12:32:00 PM 1002 Warning action=remediate qos_policy=jj1ixk4t target=ekrn.exe throttle_b...
6/17/2026 12:32:00 PM 1002 Warning action=remediate qos_policy=in5sdfx target=sentinelagent.exe t...
```

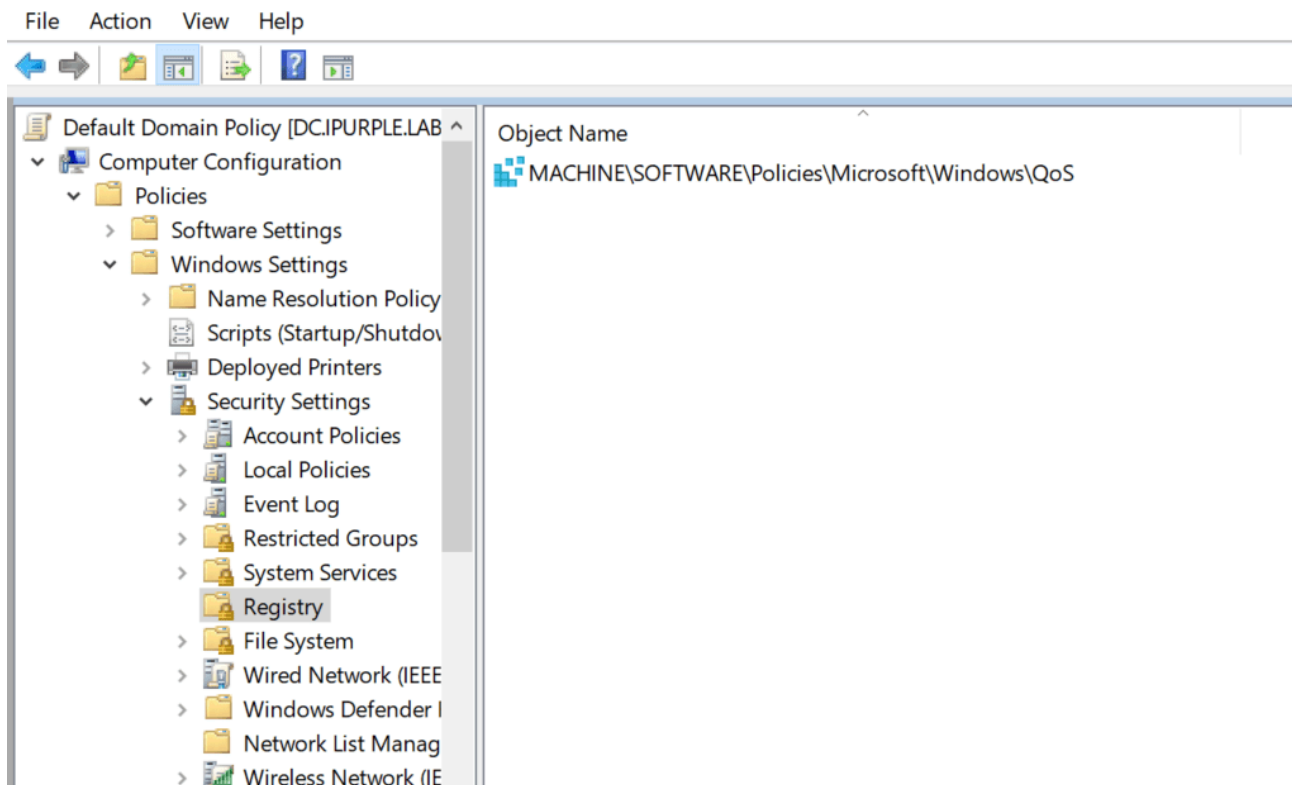
PowerShell Query – EDRUnChoker

Persistent QoS policies are stored in the registry. It should be noted that when the *ActiveStore* parameter is used, the QoS policy is stored in-memory and is not permanent. Therefore, monitoring the associated key cannot be used as a reliable detection because registry entries will not be created. However, it can be used to supplement coverage on detecting persistent arbitrary QoS policies. Persistent QoS policies are stored under the following key:



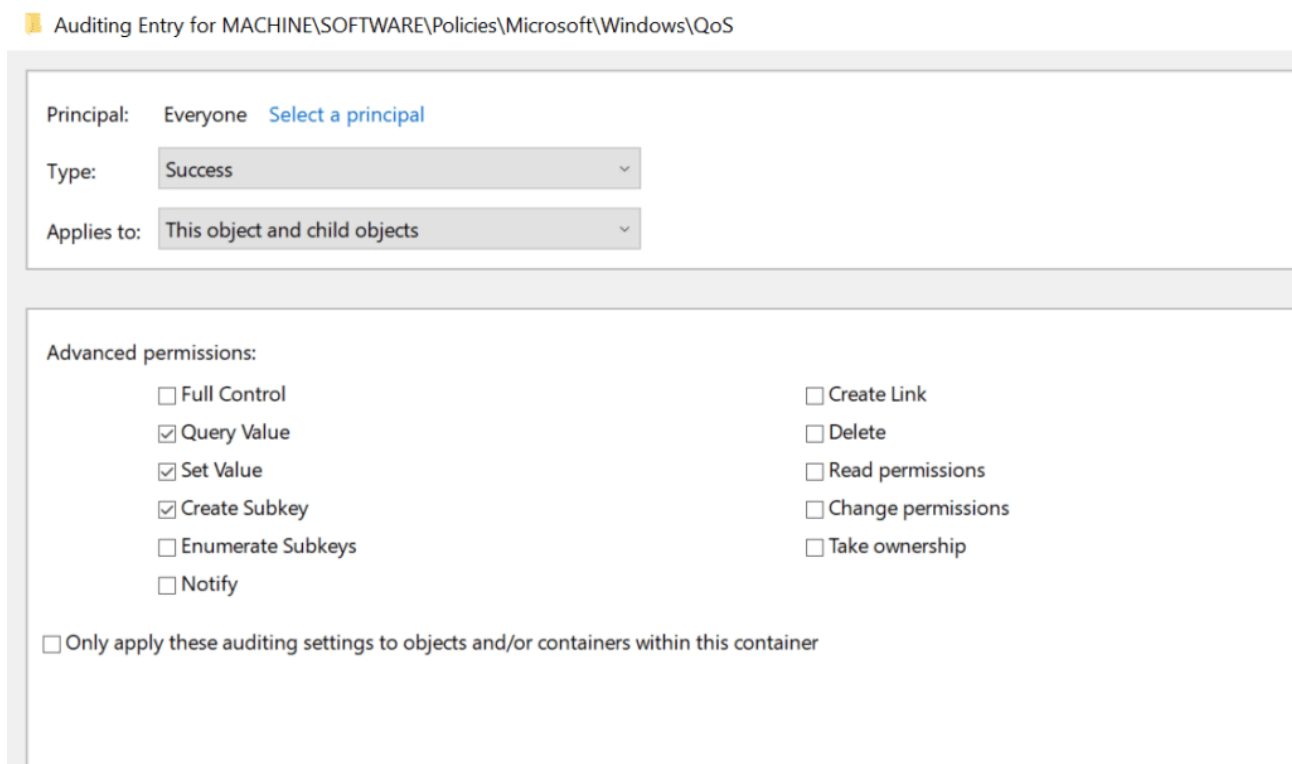
QoS Policies – Registry Key

An entry should be created under the *Registry* container in the Group Policy Management Editor.



Registry Key – QoS

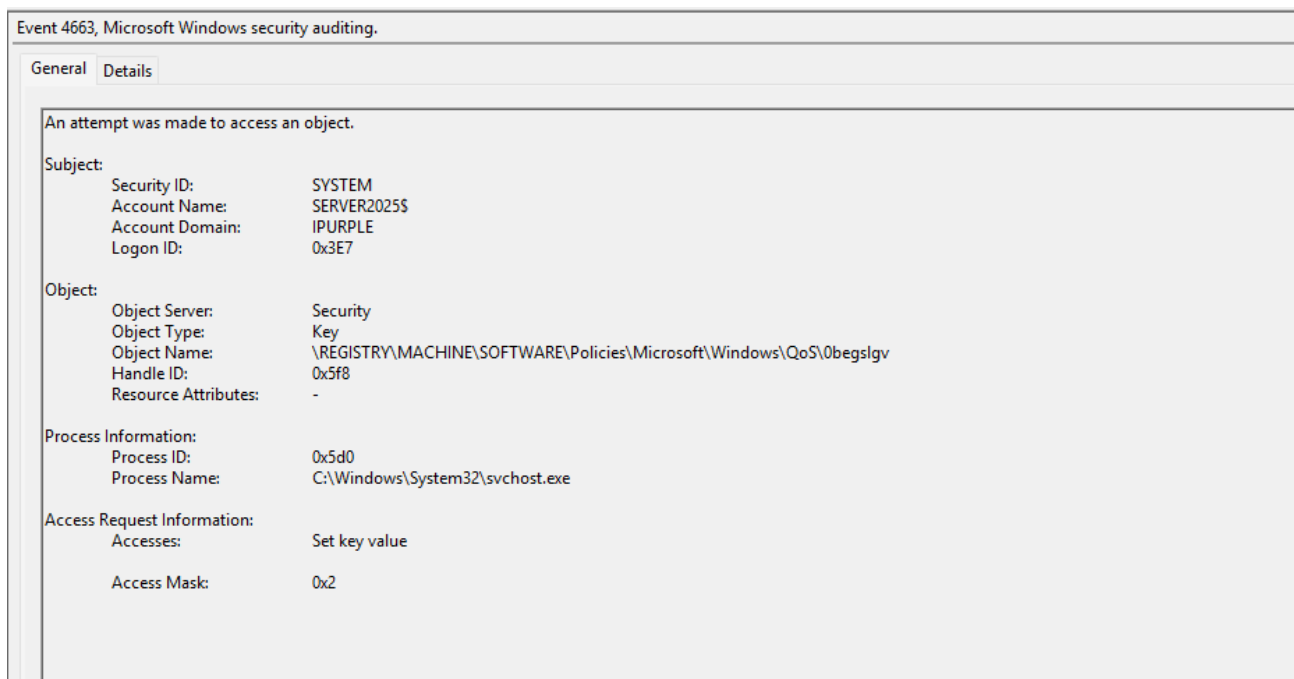
The permissions *Query Value*, *Set Value*, and *Create Subkey* should be enabled from the advanced permissions:



Auditing QoS Registry Entry

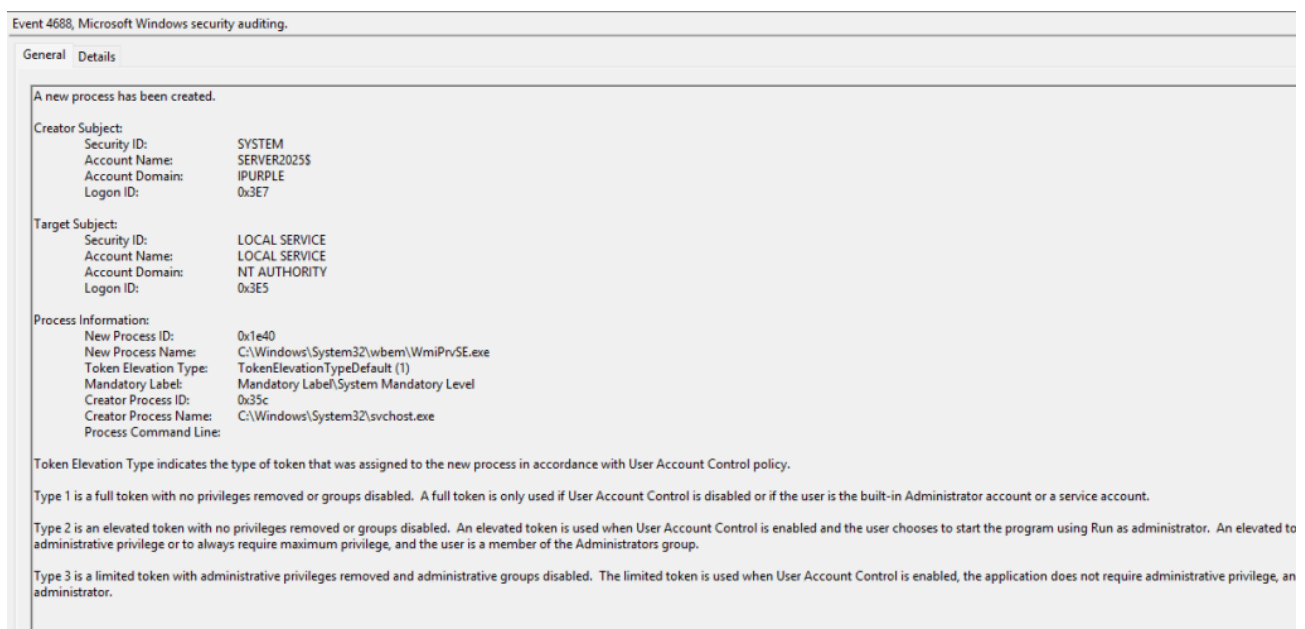
When a persistent Quality of Service policy is created, the event ID 4663 will be generated in the asset. The important field is the *Object Name* that discloses the name of the QoS

policy that could be correlated with *ThrottleRate* registry key to identify the size of the bandwidth.



Event ID 4663

During the creation of the policy, a new process (WmiPrvSE.exe) is spawned by *svchost.exe*. It should be noted that this process alone is very common, and it should not be included in the overall detection strategy, but as part of enrichment during log correlation.



Process Creation

The following SIGMA rule can be used to detect the behaviour of this technique.

```

1  title: Windows NetQoS Cim WMI Provider Started
2  id: 1b8c9b87-f70c-40a2-9d06-877fdd0042cc
3  status: experimental
4  description: Detects the NetQoS Cim WMI provider starting through
5  qoswmi.dll.
6  references:
7  - https://github.com/TwoSevenOneT/EDRChoker
8  - https://www.zerosalarium.com/2026/06/edrchoker-choking-telemetry-
9  stream-block-edr.html
10 author: Panos Gkatziroulis
11 date: 2026-06-14
12 tags:
13 - attack.stealth
14 - attack.tl685
15 logsource:
16 product: windows
17 service: wmi
18 detection:
19 selection_event:
20 EventID: 5857
21 selection_structured:
22 ProviderName: 'NetQoS Cim'
23 ProviderPath|endswith:
24 - '\System32\wbem\qoswmi.dll'
25 - '\SysWOW64\wbem\qoswmi.dll'
26 selection_message:
27 Message|contains|all:
28 - 'NetQoS Cim'
29 - 'qoswmi.dll'
30 condition: selection_event and (selection_structured or
31 selection_message)
32 falsepositives:
33 - Legitimate QoS policy administration
    - QoS policy inventory scripts
    - Group Policy or Intune-driven QoS activity
    level: medium

```

The following table summarizes the data sources and data components required to detect this technique:

Data Source	Data Component	Detects
Windows Events	4688	Process Creation
Windows Events	4663	Registry Modification
ETW	5857	NetQosCim Provider
ETW	4104	PowerShell ScriptBlock

In summary, causing EDR agents to lose connectivity by throttling the bandwidth limit prevents SOC teams from receiving alerts and responding quickly during an attack. EDR agents might still be able to protect endpoints locally, but removing the ability to receive, investigate alerts, and respond could maximize the mean-time-to-detect and mean-time-to-respond for SOC teams. Organizations should avoid creating detection rules with the EDR and focus their detection strategies towards PowerShell and WMI ETW events. Persistent QoS policies can also be detected by monitoring the registry key that stores them. In environments without deployed QoS policies, the most reliable detection involves activities related to the NetQosCim provider.